

Testprotokoll iAM-Scout

Testdurchlauf V2

1. Testobjekt: Backend

Test.Nr	Test	Erwartetes Ergebnis	Tatsächliches Ergebnis	Erfolg JA/NEIN
1.1	Starten des Backend-Services auf der Schul-VM	Der FastAPI-Service startet ohne Fehlermeldung und läuft stabil.	Der Service startet ohne Fehlermeldung und ist von der VM aus abrufbar.	ja
1.2	Erreichbarkeit des Backends von ausserhalb über das lokale Frontend	Das lokal gestartete Frontend kann erfolgreich Requests an die REST API auf der Schul-VM senden und erhält gültige Antworten.	Der ausgewählte Port blockiert den Zugriff auf das Backend	nein
1.3	Root-Route / gibt erfolgreiche Antwort	Beim Aufruf der Root-Route wird eine erfolgreiche Antwort vom Backend zurückgegeben. Dadurch wird bestätigt, dass die REST API grundsätzlich erreichbar ist und korrekt läuft.	Die Root-Route liefert eine erfolgreiche Antwort (HTTP 200), damit ist die REST-API erreichbar.	ja
1.4	API gibt Antworten im JSON-Format zurück	Die API gibt die Antwort im gültigen JSON-Format zurück. Die zurückgegebenen Daten können vom Frontend korrekt verarbeitet und angezeigt werden.	API gibt Antworten im JSON-Format zurück	ja
1.5	Abrufen von allgemeinen Stammdaten über /teams, /players und /leagues-seasons	Die API gibt die verfügbaren Teams, Spieler sowie Liga-Saison-Kombinationen korrekt im JSON-Format zurück. Die Daten können vom Frontend für Auswahlfelder und Übersichten verwendet werden.	Die Stammdaten konnten erfolgreich über die API abgerufen und im Frontend weiterverwendet werden.	ja
1.6	Abrufen von Spielerinformationen über /players/{player_id} und /player-stats/{player_id}	Bei Übergabe einer gültigen Spieler-ID gibt die API die passenden Profilinformationen sowie die zugehörigen Spielstatistiken des Spielers zurück.	Die Spielerinformationen und Statistiken wurden für eine gültige Spieler-ID korrekt zurückgegeben.	ja

1.7	Abrufen von team- und saisonbezogenen Daten über /squads, /team-league und /games	Bei Übergabe einer gültigen Team-ID und Saison gibt die API den passenden Kader, die Ligazugehörigkeit sowie die Spiele des Teams in dieser Saison zurück.	Die team- und saisonbezogenen Daten wurden korrekt über die API zurückgegeben.	ja
1.8	Abrufen von Matchdaten über /match-search, /match-overview/{match_id} und /match-player-stats/{match_id}	Die API ermöglicht die Suche nach Matches und gibt bei gültigen Parametern die passende Matchübersicht sowie die Spielerstatistiken des ausgewählten Spiels zurück.	Die Matchdaten konnten erfolgreich gesucht und die Detailinformationen korrekt abgerufen werden.	ja
1.9	Abrufen von Top-Spielern über /top-players und /league-top-players	Die API gibt die bestbewerteten Spieler anhand der übergebenen Filter wie Team, Liga, Saison oder Limit korrekt zurück.	Die Top-Spieler wurden entsprechend der gesetzten Parameter korrekt zurückgegeben.	ja
1.10	Abrufen von Clubs in der Umgebung über /clubs-in-radius	Bei Übergabe einer gültigen Postleitzahl und eines Radius gibt die API die Clubs zurück, die sich innerhalb der angegebenen Distanz befinden.	Die Clubs im angegebenen Umkreis wurden nicht korrekt zurückgegeben	nein
1.11	Abrufen von Smart-Scout-Empfehlungen über /iam-scout	Die API gibt gefilterte Spielerempfehlungen zurück. Dabei werden die gesetzten Filter wie Liga, Ort, Distanz, Alter, Positionen oder mehrere Ligen berücksichtigt.	Die Smart-Scout-Empfehlungen wurden erfolgreich zurückgegeben und die gesetzten Filter wurden berücksichtigt.	Ja (nachdem Fehler in 1.10 behoben wurde)
1.12	Aufruf einer unbekannten Route	Wenn eine nicht vorhandene Route aufgerufen wird, gibt die API eine kontrollierte Fehlermeldung zurück. Die Anwendung stürzt nicht ab und antwortet mit einem passenden HTTP-Statuscode, z. B. 404 Not Found.	Die API fängt Aufrufe an nicht vorhandene Endpunkte ab und gibt eine standardisierte Fehlerantwort mit passendem HTTP-Status 404 ohne die Anwendung abstürzen zu lassen.	ja
1.13	Aufruf einer Route mit fehlenden oder ungültigen Parametern	Wenn bei einer Route erforderliche Parameter fehlen oder ein falscher Datentyp übergeben wird, gibt die API eine kontrollierte Fehlermeldung zurück. Die Anfrage wird nicht verarbeitet und die Anwendung stürzt nicht ab.	Bei fehlenden oder ungültigen Parametern antwortet die API mit einer kontrollierten Fehlermeldung 400, verarbeitet die Anfrage nicht und bleibt stabil.	ja

2. Testobjekt: Scraping

Test.Nr	Test	Erwartetes Ergebnis	Tatsächliches Ergebnis	Erfolg JA/NEIN
2.1	Clubs Scraping: Amateur	Die Daten aller Amateur-Clubs, die in den letzten fünf Jahren in der 1. Liga oder der Promotion League gespielt haben, werden von Transfermarkt extrahiert und im Data-Ordner als CSV-Dateien gespeichert. Es wird eine Datei mit eindeutigen Clubs sowie eine Datei mit Clubs pro Saison inklusive Ligazugehörigkeit erstellt.	Alle benötigten Clubdaten wurden korrekt von Transfermarkt extrahiert und gespeichert.	JA
2.2	Clubs Scraping: Pro	Die Daten aller Profi-Clubs, die in den letzten zwei Jahren in der Super League gespielt haben, werden von Transfermarkt extrahiert und im Data-Ordner als CSV-Dateien gespeichert. Es wird eine Datei mit eindeutigen Clubs sowie eine Datei mit Clubs pro Saison inklusive Ligazugehörigkeit erstellt.	Alle benötigten Clubdaten wurden korrekt von Transfermarkt extrahiert und gespeichert.	JA
2.3	Matches Scraping: Amateur	Die Spieldaten der letzten fünf Jahre aus der 1. Liga und der Promotion League werden von Transfermarkt extrahiert und im Data-Ordner als CSV-Datei gespeichert.	Die Spieldaten der letzten fünf Jahre aus der 1. Liga und der Promotion League werden von Transfermarkt extrahiert und im Data-Ordner als CSV-Datei gespeichert.	JA
2.4	Matches Scraping: Pro	Die Spieldaten der letzten zwei Jahre aus der Super League werden von Transfermarkt extrahiert und im Data-Ordner als CSV-Datei gespeichert.	Alle benötigten Spieldaten wurden korrekt von Transfermarkt extrahiert und gespeichert.	JA
2.5	Players Scraping	Die Daten aller Spieler, die in den Katern der Mannschaften aus <code>clubs_per_season</code> enthalten sind, werden von Transfermarkt extrahiert und im Data-Ordner gespeichert.	Alle benötigten Spielerdaten wurden korrekt von Transfermarkt extrahiert und gespeichert.	JA
2.6	Spieler-Leistungsdaten Scraping	Für alle Spieler, die in den erfassten Spielen eingesetzt wurden, werden die Leistungsdaten von Transfermarkt extrahiert und im Data-Ordner gespeichert.	Mit Ausnahme der Werte <code>on_min</code> und <code>off_min</code> wurden alle Leistungsdaten korrekt extrahiert und gespeichert.	NEIN

2.7	Sofascore-Spieler Scraping	Die Daten aller Spieler, die in den letzten zwei Jahren in der Super League gespielt haben, werden von Sofascore extrahiert und im Data-Ordner gespeichert.	Die extrahierten Daten waren fehlerhaft und entsprachen nicht den erwarteten Spielerinformationen.	NEIN
2.8	Sofascore-Bewertungen Scraping	Alle Bewertungen und zusätzlichen Spielerinformationen aus den Super-League-Spielen der letzten zwei Jahre werden über Sofascore extrahiert und im Data-Ordner gespeichert.	Die extrahierten Daten waren fehlerhaft.	NEIN
2.9	Transfermarkt-Scrapes Logging	Alle Scraping-Durchläufe werden als TXT-Dateien protokolliert. Zusätzlich werden die jeweils aktuellsten Scraping-Ergebnisse als JSON-Datei gespeichert.	Alle Scraping-Durchläufe wurden korrekt als TXT-Dateien protokolliert. Die aktuellsten Scraping-Ergebnisse wurden zusätzlich als JSON-Datei gespeichert.	JA

3. Testobjekt: Transform

Test.Nr	Test	Erwartetes Ergebnis	Tatsächliches Ergebnis	Erfolg JA/NEIN
3.1	Transform-Container erkennt vorhandene CSV-Dateien	Beim Start des Transform-Containers wird geprüft, welche CSV-Dateien aus der Datenquelle vorhanden sind. Die vorhandenen Dateien werden korrekt erkannt und für die entsprechenden Transformationsschritte berücksichtigt.	Die vorhandenen CSV-Dateien wurden vom Transform-Container korrekt erkannt.	ja
3.2	Starten des Transform-Containers	Der Transform-Container startet erfolgreich und führt das zentrale Transformationsskript aus. Dabei werden die benötigten Abhängigkeiten geladen und der Transformationsprozess wird ohne Startfehler ausgeführt.	Der Transform-Container konnte erfolgreich gestartet werden und das Transformationsskript wurde ausgeführt.	ja
3.3	Transformation der Teamdaten-CSV	Die Teamdaten werden erfolgreich geladen, bereinigt und in die benötigte Datenbankstruktur	Die Teamdaten-CSV wurde erfolgreich transformiert und als	JA

		überführt. Dabei werden alte Teamnamen entfernt, fehlende PLZ-Werte behandelt, die Daten von Luzern U21 korrigiert, die PLZ-Spalte in einen Integer-Datentyp umgewandelt und nicht benötigte Spalten entfernt.	bereinigte Datei für den Datenbankimport gespeichert.	
3.4	Transformation der Teams-per-Season-CSV	Die Teams-per-Season-Daten werden erfolgreich geladen, bereinigt und in die benötigte Datenbankstruktur überführt. Dabei wird das Saisonformat korrekt von einem Jahr, z. B. 2024, in ein Saisonformat wie 24/25 umgewandelt. Zusätzlich werden die benötigten Datentypen korrekt gesetzt.	Die Teams-per-Season-CSV wurde erfolgreich transformiert und als bereinigte Datei für den Datenbankimport gespeichert.	JA
3.5	Transformation der Player-CSV	Die Spielerdaten werden erfolgreich geladen, bereinigt und in die benötigte Datenbankstruktur überführt. Dabei werden Geburtsdaten und Körpergrössen in passende Datentypen umgewandelt, ungültige Höhenwerte behandelt, Duplikate entfernt und nicht benötigte Spalten gelöscht.	Die Player-CSV wurde erfolgreich transformiert und als bereinigte Datei für den Datenbankimport gespeichert.	JA
3.6	Transformation der Matches-CSV	Die Matchdaten werden erfolgreich geladen, bereinigt und in die benötigte Datenbankstruktur überführt. Dabei werden Saison- und Datumsformat korrekt angepasst, Torwerte in passende Datentypen umgewandelt und nicht benötigte Spalten entfernt.	Die Matches-CSV wurde erfolgreich transformiert und als bereinigte Datei für den Datenbankimport gespeichert.	JA
3.7	Transformation der Squad-CSV	Die Kaderdaten werden erfolgreich geladen, bereinigt und in die benötigte Datenbankstruktur überführt. Dabei wird das Saisonformat vereinheitlicht und die ID-Spalten werden in passende Datentypen umgewandelt.	Die Squad-CSV wurde erfolgreich transformiert und als bereinigte Datei für den Datenbankimport gespeichert.	JA
3.8	Transformation der Player-Stats-CSV	Die Spielerstatistiken werden erfolgreich geladen, bereinigt und in die benötigte Datenbankstruktur überführt. Dabei werden	Die Player-Stats-CSV wurde erfolgreich transformiert und als	JA

		Karten- und Startelf-Spalten in Boolean-Werte umgewandelt, Minutenwerte korrekt behandelt und eine Rating-Spalte für die spätere Modellbewertung ergänzt.	bereinigte Datei für den Datenbankimport gespeichert.	
3.9	Anwendung des Spielerbewertungsmodells	Nach der Transformation werden die vorbereiteten Spieler- und Statistikdaten an das Bewertungsmodell übergeben. Zusätzlich werden benötigte Informationen aus der Datenbank geladen. Das Modell berechnet anschliessend Spielerbewertungen und speichert bzw. aktualisiert diese für die weitere Nutzung im System.	Das Spielerbewertungsmodell wurde erfolgreich angewendet. Die benötigten Daten konnten aus der Datenbank geladen und die Spielerbewertungen aktualisiert werden.	JA
3.10	Anwendung des Next-Season-Rating-Modells	Das Next-Season-Rating-Modell wird mit den benötigten Informationen aus der Datenbank ausgeführt. Auf dieser Basis werden Prognosen für zukünftige Spielerbewertungen berechnet und für die weitere Nutzung im System bereitgestellt.	Das Next-Season-Rating-Modell wurde erfolgreich mit den Daten aus der Datenbank ausgeführt und die Prognosewerte wurden aktualisiert.	JA

4. Testobjekt: Datenbank

Test.Nr	Test	Erwartetes Ergebnis	Tatsächliches Ergebnis	Erfolg JA/NEIN
4.1	PostgreSQL-Datenbank ist erreichbar	Die PostgreSQL-Datenbank startet erfolgreich und ist über die vorgesehenen Zugangsdaten erreichbar. Eine Verbindung zur Datenbank kann ohne Fehlermeldung hergestellt werden.	Die Datenbank startet aber ist nicht erreichbar. Der Standardport 5432 wird blockiert.	NEIN
4.2	Erstellung der Datenbankstruktur	Die PostgreSQL-Datenbank wird mit allen benötigten Tabellen, Primärschlüsseln, Fremdschlüsseln, Constraints und Indizes erstellt. Die Tabellenstruktur entspricht dem geplanten relationalen Schema und bildet Clubs,	Die Datenbankstruktur wurde erfolgreich erstellt. Alle benötigten Tabellen, Beziehungen, Constraints und Indizes waren vorhanden.	JA

		Spieler, Seasons, Matches, Kader und Spielerstatistiken korrekt ab.		
4.3	Import der Grunddaten in die PostgreSQL-Datenbank	Die transformierten Grunddaten können erfolgreich in die PostgreSQL-Datenbank importiert werden. Dabei werden die Daten in die vorgesehenen Tabellen wie clubs, players, clubs_per_season, matches, squads und player_stats geladen.	Die transformierten Grunddaten konnten erfolgreich in die vorgesehenen Tabellen der Datenbank importiert werden.	JA
4.4	Vermeidung von Duplikaten beim Datenimport	Beim Import der Daten verhindern Primärschlüssel und eindeutige Tabellenbeziehungen, dass doppelte Datensätze eingefügt werden. Bereits vorhandene Clubs, Spieler, Matches, Kaderzuordnungen oder Spielerstatistiken werden nicht mehrfach gespeichert.	Duplikate wurden durch die definierten Primärschlüssel und Beziehungen erfolgreich verhindert.	JA
4.5	Nachladen von Live-Daten in die PostgreSQL-Datenbank	Neue Daten aus dem Live-Scraping können in die bestehende PostgreSQL-Datenbank geladen werden. Dabei werden vorhandene Daten ergänzt, ohne die bestehende Datenbankstruktur oder bereits gespeicherte Datensätze zu beschädigen.	Die Live-Daten konnten erfolgreich in die bestehende Datenbank nachgeladen werden.	JA
4.6	Ausführung von Datenbankabfragen für Backend und API	Die benötigten SQL-Abfragen können erfolgreich auf der PostgreSQL-Datenbank ausgeführt werden. Dabei werden die passenden Daten für Teams, Spieler, Kader, Matches, Statistiken und Scout-Funktionen korrekt zurückgegeben und können vom Backend weiterverarbeitet werden.	Die Datenbankabfragen wurden erfolgreich ausgeführt und lieferten die erwarteten Resultate für die Backend-Routen.	JA
4.7	Prüfung der Datenintegrität durch Constraints und Fremdschlüssel	Die Datenbank verhindert ungültige oder inkonsistente Einträge. Dazu gehören z. B. negative Torwerte, Minuten ausserhalb des erlaubten Bereichs, Matches mit gleichem Heim- und Auswärtsteam oder Verweise auf nicht vorhandene Clubs, Spieler oder Matches.	Ungültige oder inkonsistente Datensätze wurden durch die definierten Constraints und Fremdschlüssel verhindert.	JA

5. Testobjekt: Live

Test.Nr	Test	Erwartetes Ergebnis	Tatsächliches Ergebnis	Erfolg JA/NEIN
5.1	Airflow: jährlicher Dag	Der jährliche DAG wird jedes Jahr am 1. August automatisch gestartet.	Der jährliche DAG wurde wie erwartet am 1. August gestartet.	JA
5.2	Airflow: wöchentlicher Dag	Der wöchentliche DAG wird jeden Montag automatisch gestartet.	Der wöchentliche DAG wurde wie erwartet am Montag gestartet.	JA
5.3	Jährlich: aktuelle Saison aktualisieren	In <code>last_scrapes.json</code> wird die Saison 2025 als aktuelle Saison gespeichert.	In <code>last_scrapes.json</code> wurde die Saison 2025 korrekt als aktuelle Saison gespeichert.	JA
5.4	Jährlich: scraper/clubs ausführen	Alle Amateurclubs der neuen Saison werden von Transfermarkt extrahiert und gespeichert. Dabei werden sowohl eine Datei mit eindeutigen Clubs als auch eine Datei mit Clubs pro Saison inklusive Ligazugehörigkeit erstellt.	Alle Amateurclubs der neuen Saison wurden korrekt von Transfermarkt extrahiert und gespeichert. Beide Dateien wurden erfolgreich erstellt.	JA
5.5	Wöchentlich: <code>scraper/matches</code> ausführen	Alle Daten der neu hinzugekommenen Spiele werden extrahiert und gespeichert.	Alle Daten der neuen Spiele wurden erfolgreich extrahiert und gespeichert.	JA
5.6	Wöchentlich: <code>scraper/player</code> ausführen	Alle Daten der neu hinzugekommenen Spieler werden extrahiert und gespeichert.	Spieler, die bisher nicht in der Datenbank vorhanden waren, konnten nicht extrahiert werden.	NEIN
5.7	Wöchentlich: <code>scraper/player_stats</code> ausführen	Alle Leistungsdaten der neuen Spiele werden extrahiert und gespeichert.	Die erzeugte CSV-Datei enthielt keine Datensätze.	NEIN
5.8	Jährlich: Transform	Alle Daten werden transformiert und korrekt gespeichert.	Alle Daten wurden erfolgreich transformiert und korrekt gespeichert.	JA

5.9	Wöchentlich: Transfrom	Alle Daten werden transformiert und korrekt gespeichert.	Alle Daten wurden erfolgreich transformiert und korrekt gespeichert.	JA
5.10	Jährlich: Load	Nach der Transformation werden alle Daten in die Datenbank geladen.	Die Daten konnten nicht in die Datenbank geladen werden.	NEIN
5.11	Wöchentlich: Load	Nach der Transformation werden alle Daten in die Datenbank geladen.	Die Daten konnten nicht in die Datenbank geladen werden.	NEIN
5.12	Wöchentlich: Bewertung-Modell	Das Bewertungsmodell wird auf die neuen Leistungsdaten angewendet. Jeder Spieler erhält für jedes Spiel eine Bewertung.	Jeder Spieler hat für jedes Spiel eine Bewertung erhalten.	JA
5.13	Wöchentlich: Recommender-Modell	Das Recommender-Modell wird auf alle bisherigen und neuen Leistungsdaten angewendet. Die Vorhersagebewertung wird aktualisiert.	Die Vorhersagebewertung wurde anhand der neuen Leistungsdaten aktualisiert. Die Liga-Differenz wurde jedoch nicht korrekt angewendet.	NEIN

6. Testobjekt: Frontend

Test.Nr	Test	Erwartetes Ergebnis	Tatsächliches Ergebnis	Erfolg JA/NEIN
6.1	Starten der Streamlit-App	Die Streamlit-App startet erfolgreich und ist im Browser erreichbar. Die Startseite wird ohne Fehlermeldung geladen.	Die Streamlit-App konnte erfolgreich gestartet und im Browser geöffnet werden.	JA
6.2	Navigation zwischen den Seiten	Der Benutzer kann über die Navigation zwischen den verschiedenen Seiten der Streamlit-App wechseln. Die ausgewählte Seite wird korrekt geladen und angezeigt.	Die Navigation zwischen den verschiedenen Seiten funktionierte erfolgreich. Die Seiten wurden korrekt geladen.	JA
6.3	Verbindung zwischen Frontend und Backend	Das Streamlit-Frontend kann erfolgreich Anfragen an die REST API senden und die erhaltenen Daten in der App anzeigen.	Das Frontend konnte erfolgreich Daten vom Backend abrufen und anzeigen.	JA
6.4	Auswahl von Team und Saison in Team Insights	Der Benutzer kann ein Team und eine Saison auswählen. Die Seite lädt danach die passenden	Team und Saison konnten erfolgreich ausgewählt werden und	JA

		Daten für das ausgewählte Team und die gewählte Saison.	die passenden Daten wurden angezeigt.	
6.5	Anzeige der Team-Informationen in Team Insights	Die Seite zeigt die wichtigsten Informationen zum ausgewählten Team an, darunter Kaderübersicht, Topspieler und Spiele. Falls für eine Saison keine Daten vorhanden sind, wird eine verständliche Meldung angezeigt.	Die Teamdaten wurden korrekt angezeigt. Bei fehlenden Daten wurde eine Warnmeldung ausgegeben.	JA
6.6	Auswahl eines Spielers in Player Insights	Der Benutzer kann einen Spieler auswählen. Nach der Auswahl werden die passenden Spielerinformationen und Leistungsdaten über das Backend geladen.	Der Spieler konnte erfolgreich ausgewählt werden und die passenden Daten wurden geladen.	JA
6.7	Anzeige der Spielerinformationen und Leistungsdaten	Die Seite zeigt die wichtigsten Informationen zum ausgewählten Spieler an, darunter Name, Nationalität, Geburtsdatum, Alter, Grösse, Position sowie Leistungsdaten wie Anzahl Spiele und durchschnittliches Rating. Falls keine Daten vorhanden sind, wird eine verständliche Meldung angezeigt.	Die Spielerinformationen und Leistungsdaten wurden korrekt angezeigt. Bei fehlenden Daten wurde eine Warnmeldung ausgegeben.	JA
6.8	Auswahl eines Matches in Match Insights	Der Benutzer kann ein Match entweder über die Match-ID oder über die Auswahl von zwei Teams suchen und auswählen. Falls kein passendes Match gefunden wird, erscheint eine verständliche Meldung.	Matches konnten erfolgreich über Match-ID oder Teamauswahl gesucht und ausgewählt werden. Bei fehlenden Resultaten wurde eine Meldung angezeigt.	JA
6.9	Anzeige der Matchübersicht und Spielerstatistiken	Nach Auswahl eines Matches zeigt die Seite die wichtigsten Matchinformationen wie Teams, Resultat, Datum, Liga und Saison an. Zusätzlich werden die Spielerstatistiken der Heim- und Auswärtsmannschaft getrennt dargestellt.	Die Matchübersicht und die Spielerstatistiken wurden korrekt geladen und angezeigt.	JA
6.10	Auswahl von Liga und Saison in League Insights	Der Benutzer kann eine Liga auswählen. Danach werden nur die verfügbaren Saisons dieser Liga angezeigt und auswählbar gemacht. Falls keine Liga- oder Saisondaten vorhanden sind, erscheint eine verständliche Meldung.	Liga und Saison konnten erfolgreich ausgewählt werden. Bei fehlenden Daten wurde eine Meldung angezeigt.	JA

	Anzeige der Topspieler einer Liga	Für die ausgewählte Liga und Saison werden die bestbewerteten Spieler der Liga geladen und in einer Tabelle angezeigt. Falls keine Topspieler-Daten vorhanden sind, erscheint eine verständliche Meldung.	Die Topspieler der ausgewählten Liga und Saison wurden korrekt geladen und angezeigt.	JA
6.11	Setzen von Filtern im Smart Scout	Der Benutzer kann Filter wie Alter, Liga, Ort, Position und Entfernung setzen. Nach dem Anwenden der Filter werden diese gespeichert und für die Spielersuche berücksichtigt.	Manche Filter (Alter und Region) funktionieren nicht	NEIN
6.12	Anzeige der gefilterten Spieler im Smart Scout	In der Spielerdatenbank werden die Spieler passend zu den gesetzten Filtern angezeigt. Die Resultate enthalten relevante Informationen wie Spielernamen, Position, Club, Ort und Alter. Falls keine passenden Spieler gefunden werden, erscheint eine verständliche Meldung.	Die gefilterten Spieler wurden korrekt angezeigt. Bei leeren Resultaten wurde eine passende Meldung ausgegeben.	JA

Testdurchlauf V2

Test.Nr	Test	Erwartetes Ergebnis	Tatsächliches Ergebnis	Erfolg JA/NEIN
1.2	Erreichbarkeit des Backends von ausserhalb über das lokale Frontend	Der FastAPI-Service startet ohne Fehlermeldung und läuft stabil.	Mit dem neuen Port ist das Backend von ausserhalb der VM erreichbar.	Ja
1.10	Abrufen von Clubs in der Umgebung über /clubs-in-radius	Bei Übergabe einer gültigen Postleitzahl und eines Radius gibt die API die Clubs zurück, die sich innerhalb der angegebenen Distanz befinden.	Nach Anpassen der Filterlogik werden die Clubs richtig nach Radius gefiltert und zurückgegeben	JA
2.6	Spieler-Leistungsdaten Scraping	on_min und off_min werden korrekt von Transfermarkt extrahiert.	on_min und off_min wurden korrekt extrahiert und gespeichert. Es fehlten jedoch weiterhin Leistungsdaten einzelner Spieler.	NEIN

2.7	Sofascore-Spieler Scraping	Nach der Implementierung mit Playwright wird der Test erneut durchgeführt. Alle benötigten Spielerdaten werden korrekt extrahiert und gespeichert.	Alle benötigten Spielerdaten wurden korrekt extrahiert und gespeichert.	JA
2.8	Sofascore-Bewertungen Scraping	Der Test wird über eine alternative Sofascore-Seite erneut durchgeführt. Die korrekten Bewertungsdaten werden ausgelesen und gespeichert.	Bei fehlenden Bewertungen wurden teilweise Uhrzeiten oder Resultate fälschlicherweise als Bewertung interpretiert.	NEIN
5.6	Wöchentlich: <code>scraper/player</code> ausführen	Nach der Implementierung einer neuen Funktion werden auch Spieler berücksichtigt, die bisher nicht in der Datenbank vorhanden waren.	Die Daten der bisher nicht vorhandenen Spieler wurden erfolgreich extrahiert und gespeichert.	JA
5.7	Wöchentlich: <code>scraper/player_stats</code> ausführen	Die Fehlerursache wird analysiert.	Die Ursache liegt nicht in der Live-Logik, sondern im Modul <code>scraper/player_stats</code> . Dieses muss erneut getestet werden.	NEIN
5.10	Jährlich: Load Mit neuem Container (<code>update_data</code>) Test nochmals durchgeführt	Mit dem neuen Container <code>update_data</code> wird der Test erneut durchgeführt. Alle Daten werden in die Datenbank geladen.	Alle Daten wurden erfolgreich in die Datenbank geladen.	JA
5.11	Wöchentlich: Load Mit neuem Container (<code>update_data</code>) Test nochmals durchgeführt	Mit dem neuen Container <code>update_data</code> wird der Test erneut durchgeführt. Alle Daten werden in die Datenbank geladen.	Alle Daten wurden erfolgreich in die Datenbank geladen.	JA
5.13	Wöchentlich: Recommender-Modell	Die Liga-Differenz wird korrekt angewendet.	Die Liga-Differenz wurde korrekt angewendet.	JA
4.1	PostgreSQL-Datenbank ist erreichbar	Die PostgreSQL-Datenbank startet erfolgreich und ist über die vorgesehenen Zugangsdaten erreichbar. Eine Verbindung zur Datenbank kann ohne Fehlermeldung hergestellt werden.	Nach abändern des Ports klappt der Zugriff	JA
6.11	Setzen von Filtern im Smart Scout	Der Benutzer kann Filter wie Alter, Liga, Ort, Position und Entfernung setzen. Nach dem	Alle Filter funktionieren korrekt	JA

		Anwenden der Filter werden diese gespeichert und für die Spiellersuche berücksichtigt.		
--	--	--	--	--

Testdurchlauf V3

Test.Nr	Test	Erwartetes Ergebnis	Tatsächliches Ergebnis	Erfolg JA/NEIN
2.6	Spieler-Leistungsdaten Scraping	Die Leistungsdaten aller Spieler werden von Transfermarkt extrahiert.	Auch die zuvor fehlenden Leistungsdaten einzelner Spieler wurden erfolgreich extrahiert.	JA
2.8	Sofascore-Bewertungen Scraping	Bei fehlenden Bewertungen wird der Wert NaN zurückgegeben.	Bei fehlenden Bewertungen wurde korrekt NaN zurückgegeben. Es wurden keine Uhrzeiten oder Resultate mehr fälschlicherweise als Bewertung interpretiert.	JA
2.6/3.7	Spieler-Leistungsdaten Scraping	Die Leistungsdaten können mit einem Playwright-Client erneut extrahiert werden.	Auch mit Playwright konnten die Daten nicht mehr extrahiert werden. Ursache ist eine Änderung auf der Transfermarkt-Webseite, wodurch nur leeres HTML zurückgegeben wird. Der Fehler kann aktuell nicht behoben werden.	NEIN Fehler kann zurzeit nicht behoben werden